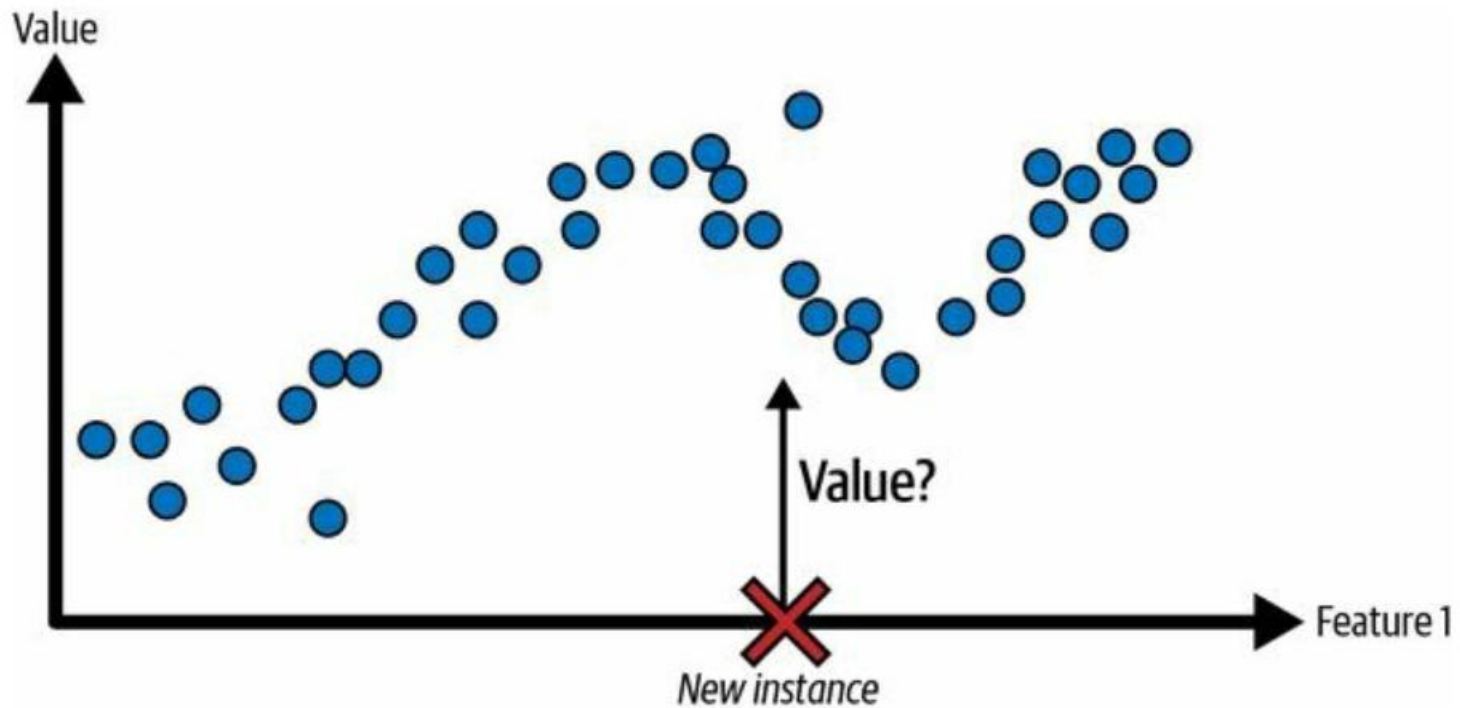


Supervised Learning: Regression

A typical task is to predict a target numeric value, such as the price of a car, given a set of features (mileage, age, brand, etc.). This sort of task is called **regression**.



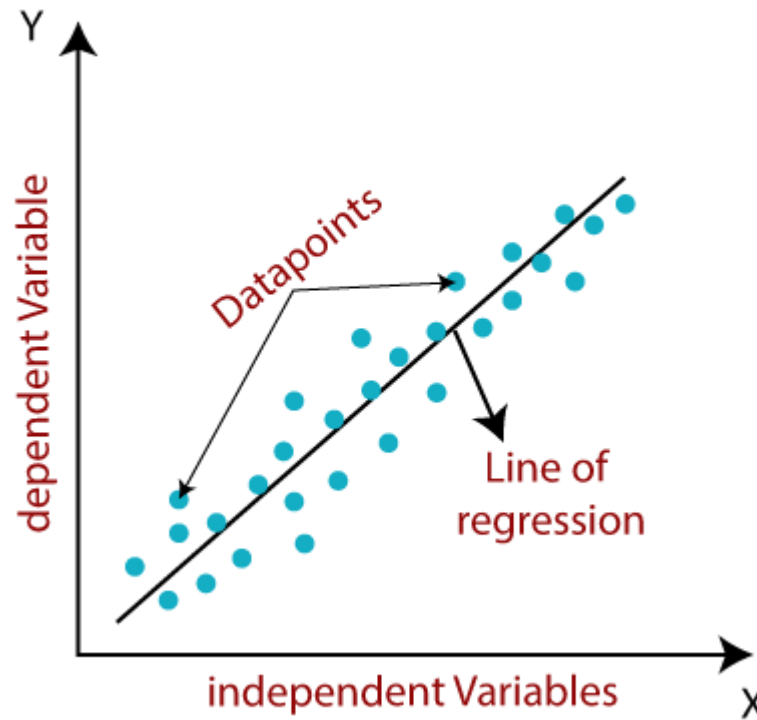
A regression problem: predict a value, given an input feature (there are usually multiple input features, and sometimes multiple output values)

Regression

- Simple Linear Regression
- Multiple Linear Regression
- Polynomial Linear Regression
- Logistic Regression

Linear Regression

Linear regression is a **supervised** machine learning algorithm that models the linear relationship between a **dependent** variable and one or more **independent** variables by fitting a straight line to the observed data.



Linear Regression

Linear regression is a **supervised** machine learning algorithm that models the linear relationship between a **dependent** variable and one or more **independent** variables by fitting a straight line to the observed data.

When there is a single independent variable, it is referred to as **Simple Linear Regression**, whereas with multiple independent variables, it is called **Multiple Linear Regression**.

Polynomial Linear Regression is a form of regression where the relationship between the independent variable(s) and the dependent variable is modeled as a **polynomial**, but it is still considered a type of linear regression because the model is linear in terms of the coefficients.

Simple Linear Regression

This is the most basic form of linear regression, involving a single independent variable and a single dependent variable. The equation for simple linear regression is: $y = w_0 + w_1x$

where

- y is the dependent variable
- x is the independent variable
- w_0 is the intercept
- w_1 is the slope

Multiple Linear Regression

This involves multiple independent variables and a single dependent variable.
The equation for simple linear regression is:

$$y = w_0 + w_1x_1 + w_2x_2 + w_3x_3 + \dots + w_n x_n$$

where

- y is the dependent variable
- $x_1, x_2, \dots, x_n$ are the independent variables
- w_0 is the intercept
- $w_1, w_2, \dots, w_n$ are the slopes

We can also write:

$$y = w_0x_0 + w_1x_1 + w_2x_2 + w_3x_3 + \dots + w_n x_n \text{ where } x_0 = 1$$

Linear Regression

The objective of the algorithm is to determine the **best-fit line equation** that can predict values based on the independent variables.

In regression, a dataset with **X** and **Y** values is provided, and these values are used to **train** a function. Once trained, this function can be applied to predict **Y** for an **unknown X**.

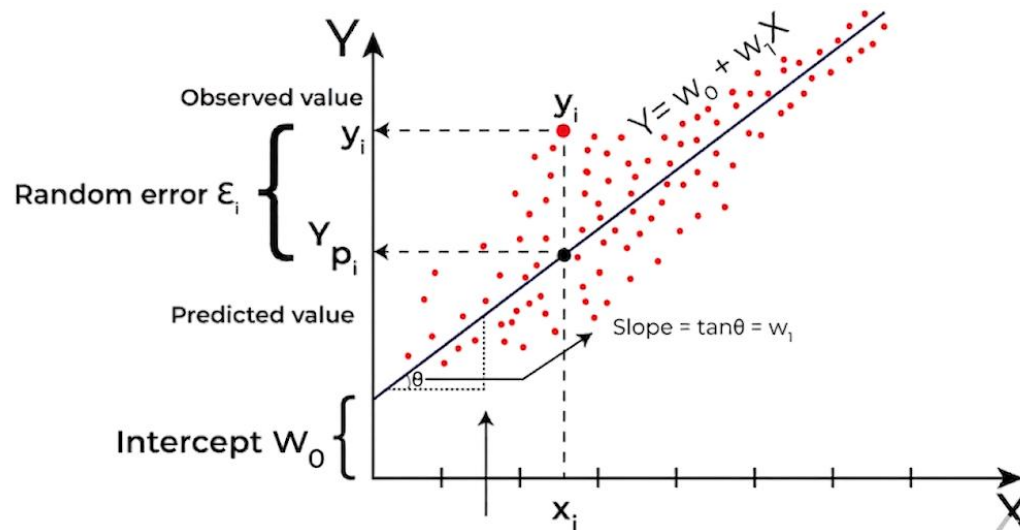
In regression, the goal is to estimate the value of **Y**, meaning a function is needed that predicts the continuous **Y** value given **X** as the independent variable(s).

Linear Regression

What is the best Fit Line?

The main goal of using linear regression is to find the best-fit line, which **minimizes the error** between the predicted and actual values. The best-fit line will have the least amount of error.

The equation of the best-fit line represents the relationship between the dependent and independent variables, with the slope indicating how much the dependent variable changes in response to a unit change in the independent variable(s).



Linear Regression

Consider a dataset

	x_1	x_2	$\dots$	x_n	y
$i = 1$	$x(1, 1)$	$x(1, 2)$	$\dots$	$x(1, n)$	$y(1)$
$i = 2$	$x(2, 1)$	$x(2, 2)$	$\dots$	$x(2, n)$	$y(2)$
$\vdots$	$\vdots$	$\vdots$	$\ddots$	$\vdots$	$\vdots$
$i = m$	$x(m, 1)$	$x(m, 2)$	$\dots$	$x(m, n)$	$y(m)$

Explanation

- $i = 1, 2, \dots, m \rightarrow$ indexes **examples (rows)**
- $j = 1, 2, \dots, n \rightarrow$ indexes **features (columns)**
- $x(i, j) \rightarrow$ value of the **j-th feature of the i-th example**
- $y(i) \rightarrow$ **target/output** for the i-th example

Linear Regression

The dataset in Matrix Form

$$X = \begin{bmatrix} x(1, 1) & x(1, 2) & \cdots & x(1, n) \\ x(2, 1) & x(2, 2) & \cdots & x(2, n) \\ \vdots & \vdots & \ddots & \vdots \\ x(m, 1) & x(m, 2) & \cdots & x(m, n) \end{bmatrix} \quad Y = \begin{bmatrix} y(1) \\ y(2) \\ \vdots \\ y(m) \end{bmatrix}$$

Feature Matrix (with bias)

$$X = \begin{bmatrix} 1 & x(1, 1) & x(1, 2) & \cdots & x(1, n) \\ 1 & x(2, 1) & x(2, 2) & \cdots & x(2, n) \\ \vdots & \vdots & \vdots & \ddots & \vdots \\ 1 & x(m, 1) & x(m, 2) & \cdots & x(m, n) \end{bmatrix}$$

Linear Regression

Notations

$$X = \begin{bmatrix} x_{1,0} & x_{1,1} & \dots x_{1,n} \\ x_{2,0} & x_{2,1} & \dots x_{2,n} \\ \vdots & \vdots & \vdots \\ x_{m,0} & x_{m,1} & \dots x_{m,n} \end{bmatrix}$$

$$X^T = \begin{bmatrix} x_{1,0} & x_{2,0} & \dots x_{m,0} \\ x_{1,1} & x_{2,1} & \dots x_{m,1} \\ \vdots & \vdots & \vdots \\ x_{1,n} & x_{2,n} & \dots x_{m,n} \end{bmatrix}$$

Remember

$$y = w_0x_0 + w_1x_1 + w_2x_2 + w_3x_3 + \dots + w_nx_n$$

where $x_0 = 1$

For i th row in the dataset, we can write the prediction as,

$$\hat{y}_i = w_0x_{i,0} + w_1x_{i,1} + \dots + w_nx_{i,n}$$

Let we want to build a linear regression model for the dataset to predict target value $\hat{y}_i$ with weights $\mathbf{w} = (w_0, w_1, w_2, \dots, w_n)$ and inputs $\mathbf{X}_i = (x_{i,0}, x_{i,1}, \dots, x_{i,n})$

$$\hat{y}_i = w_0x_{i,0} + w_1x_{i,1} + \dots + w_nx_{i,n} = \mathbf{w} \cdot \mathbf{X}_i$$

where $x_{i,0} = 1$ and $i = 1, m$

Linear Regression

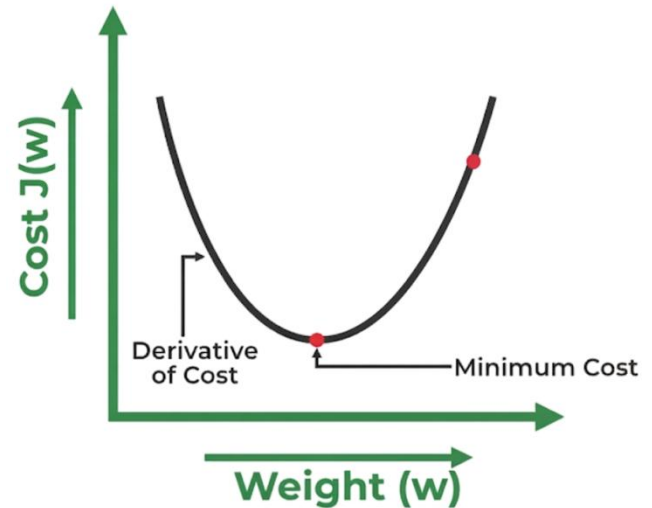
How to pick right w

Want to find the weight vector $\mathbf{w} = (w_0, w_1, w_2, \dots, w_n)$ such that the error is following error function is minimized:

$$J(\mathbf{w}) = \frac{1}{2} \sum_{i=1}^m (\hat{y}_i - y_i)^2$$

This is Sum-of-squares error function with m examples.

$$\begin{aligned} \frac{\partial}{\partial w_j} J(\mathbf{w}) &= \frac{1}{2} \sum_{i=1}^m 2(\hat{y}_i - y_i) \frac{\partial}{\partial w_j} \hat{y}_i \\ &= \sum_{i=1}^m (\hat{y}_i - y_i) \frac{\partial}{\partial w_j} (w_0 x_{i,0} + w_1 x_{i,1} + \dots + w_j x_{i,j} + \dots + w_n x_{i,n}) \\ \Rightarrow \frac{\partial}{\partial w_j} J(\mathbf{w}) &= \sum_{i=1}^m (\hat{y}_i - y_i) x_{i,j} \end{aligned}$$



Linear Regression

How to pick right w

Replacing the j by 1 to n and expanding the right side sum, we get

$$\frac{\partial}{\partial w_0} J(w) = (\hat{y}_1 - y_1)x_{1,0} + (\hat{y}_2 - y_2)x_{2,0} + \cdots + (\hat{y}_m - y_m)x_{m,0}$$

$$\frac{\partial}{\partial w_1} J(w) = (\hat{y}_1 - y_1)x_{1,1} + (\hat{y}_2 - y_2)x_{2,1} + \cdots + (\hat{y}_m - y_m)x_{m,1}$$

$\vdots$

$$\frac{\partial}{\partial w_n} J(w) = (\hat{y}_1 - y_1)x_{1,n} + (\hat{y}_2 - y_2)x_{2,n} + \cdots + (\hat{y}_m - y_m)x_{m,n}$$

Linear Regression

How to pick right $\mathbf{w}$

Compute $\mathbf{w}$ such that $J(\mathbf{w})$ is minimal, that is such that: $\frac{\partial}{\partial w_j} J(\mathbf{w}) = 0$ for $j = 1, n$

We get the following system of linear equations:

$$\begin{aligned}(\hat{y}_1 - y_1)x_{1,0} + (\hat{y}_2 - y_2)x_{2,0} + \cdots + (\hat{y}_m - y_m)x_{m,0} &= 0 \\(\hat{y}_1 - y_1)x_{1,1} + (\hat{y}_2 - y_2)x_{2,1} + \cdots + (\hat{y}_m - y_m)x_{m,1} &= 0 \\&\vdots \\(\hat{y}_1 - y_1)x_{1,n} + (\hat{y}_2 - y_2)x_{2,n} + \cdots + (\hat{y}_m - y_m)x_{m,n} &= 0\end{aligned}$$

Simplifying the equations, we get:

$$\begin{aligned}[\hat{y}_1 x_{1,0} + \hat{y}_2 x_{2,0} + \cdots + \hat{y}_m x_{m,0}] - [x_{1,0} y_1 + x_{2,0} y_2 + \cdots + x_{m,0} y_m] &= 0 \\[\hat{y}_1 x_{1,1} + \hat{y}_2 x_{2,1} + \cdots + \hat{y}_m x_{m,1}] - [x_{1,1} y_1 + x_{2,1} y_2 + \cdots + x_{m,1} y_m] &= 0 \\&\vdots \\[\hat{y}_1 x_{1,n} + \hat{y}_2 x_{2,n} + \cdots + \hat{y}_m x_{m,n}] - [x_{1,n} y_1 + x_{2,n} y_2 + \cdots + x_{m,n} y_m] &= 0\end{aligned}$$

Linear Regression

How to pick right w

After applying, $\hat{y}_i = w \cdot X_i$

$$[(w \cdot X_1)x_{1,0} + (w \cdot X_2)x_{2,0} + \cdots + (w \cdot X_m)x_{m,0}] - [x_{1,0}y_1 + x_{2,0}y_2 + \cdots + x_{m,0}y_m] = 0$$

$$[(w \cdot X_1)x_{1,1} + (w \cdot X_2)x_{2,1} + \cdots + (w \cdot X_m)x_{m,1}] - [x_{1,1}y_1 + x_{2,1}y_2 + \cdots + x_{m,1}y_m] = 0$$

$\vdots$

$$[(w \cdot X_1)x_{1,n} + (w \cdot X_2)x_{2,n} + \cdots + (w \cdot X_m)x_{m,n}] - [x_{1,n}y_1 + x_{2,n}y_2 + \cdots + x_{m,n}y_m] = 0$$

Linear Regression

How to pick right w

The following system linear equation can be derived:

$$\begin{aligned}
 & [w.X_1 \quad w.X_2 \quad \dots \quad w.X_m] \begin{bmatrix} x_{1,0} \\ x_{2,0} \\ \vdots \\ x_{m,0} \end{bmatrix} - [x_{1,0} \quad x_{2,0} \quad \dots \quad x_{m,0}] \begin{bmatrix} y_1 \\ y_2 \\ \vdots \\ y_m \end{bmatrix} = 0 \\
 & [w.X_1 \quad w.X_2 \quad \dots \quad w.X_m] \begin{bmatrix} x_{1,1} \\ x_{2,1} \\ \vdots \\ x_{m,1} \end{bmatrix} - [x_{1,1} \quad x_{2,1} \quad \dots \quad x_{m,1}] \begin{bmatrix} y_1 \\ y_2 \\ \vdots \\ y_m \end{bmatrix} = 0 \\
 & \vdots \\
 & [w.X_1 \quad w.X_2 \quad \dots \quad w.X_m] \begin{bmatrix} x_{1,n} \\ x_{2,n} \\ \vdots \\ x_{m,n} \end{bmatrix} - [x_{1,n} \quad x_{2,n} \quad \dots \quad x_{m,n}] \begin{bmatrix} y_1 \\ y_2 \\ \vdots \\ y_m \end{bmatrix} = 0
 \end{aligned}$$

Linear Regression

How to pick right w

The simplified form of the system is:

$$[w \cdot X_1 \quad w \cdot X_2 \quad \dots \quad w \cdot X_m] \begin{bmatrix} x_{1,0} & x_{1,1} & \dots & x_{1,n} \\ x_{2,0} & x_{2,1} & \dots & x_{2,n} \\ \vdots & \vdots & & \vdots \\ x_{m,0} & x_{m,1} & \dots & x_{m,n} \end{bmatrix} - \begin{bmatrix} x_{1,0} & x_{2,0} & \dots & x_{m,0} \\ x_{1,1} & x_{2,1} & \dots & x_{m,1} \\ \vdots & \vdots & & \vdots \\ x_{1,n} & x_{2,n} & \dots & x_{m,n} \end{bmatrix} \begin{bmatrix} y_1 \\ y_2 \\ \vdots \\ y_m \end{bmatrix} = 0$$

$$\Rightarrow [w \cdot X_1 \quad w \cdot X_2 \quad \dots \quad w \cdot X_m] \mathbf{X} - \mathbf{X}^T \mathbf{Y} = 0$$

$$\Rightarrow [w_0 \quad w_1 \quad \dots \quad w_n] \begin{bmatrix} x_{1,0} & x_{2,0} & \dots & x_{m,0} \\ x_{1,1} & x_{2,1} & \dots & x_{m,1} \\ \vdots & \vdots & & \vdots \\ x_{1,n} & x_{2,n} & \dots & x_{m,n} \end{bmatrix} \mathbf{X} - \mathbf{X}^T \mathbf{Y} = 0$$

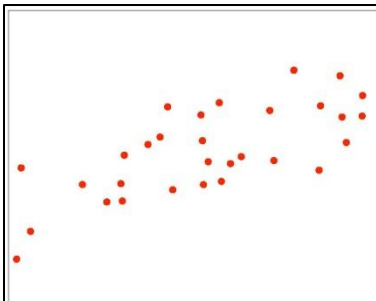
$$\Rightarrow w \mathbf{X}^T \mathbf{X} - \mathbf{X}^T \mathbf{Y} = 0$$

$$\Rightarrow w = (\mathbf{X}^T \mathbf{X})^{-1} \mathbf{X}^T \mathbf{Y}$$

The inverse exists if the columns of $\mathbf{X}$ are linearly independent.

Linear Regression

An example: $w = (X^T X)^{-1} X^T Y$



x	y
0.86	2.49
0.09	0.83
-0.85	-0.25
0.87	3.10
-0.44	0.87
-0.43	0.02
-1.10	-0.12
0.40	1.81
-0.96	-0.83
0.17	0.43

$$X = \begin{bmatrix} 1 & 0.86 \\ 1 & 0.09 \\ 1 & -0.85 \\ 1 & 0.87 \\ 1 & -0.44 \\ 1 & -0.43 \\ 1 & -1.10 \\ 1 & 0.40 \\ 1 & -0.96 \\ 1 & 0.17 \end{bmatrix}$$

$$Y = \begin{bmatrix} 2.49 \\ 0.83 \\ -0.25 \\ 3.10 \\ 0.87 \\ 0.02 \\ -0.12 \\ 1.81 \\ -0.83 \\ 0.43 \end{bmatrix}$$

Linear Regression

An example

$$X^T X =$$

$$\begin{bmatrix} 1 & 1 & 1 & 1 & 1 & 1 & 1 & 1 & 1 & 1 \\ 0.86 & 0.09 & -0.85 & 0.87 & -0.44 & -0.43 & -1.10 & 0.40 & -0.96 & 0.17 \end{bmatrix} \times \begin{bmatrix} 1 & 0.86 \\ 1 & 0.09 \\ 1 & -0.85 \\ 1 & 0.87 \\ 1 & -0.44 \\ 1 & -0.43 \\ 1 & -1.10 \\ 1 & 0.40 \\ 1 & -0.96 \\ 1 & 0.17 \end{bmatrix}$$

$$= \begin{bmatrix} 10 & -1.39 \\ -1.39 & 4.9261 \end{bmatrix}$$

Linear Regression

An example

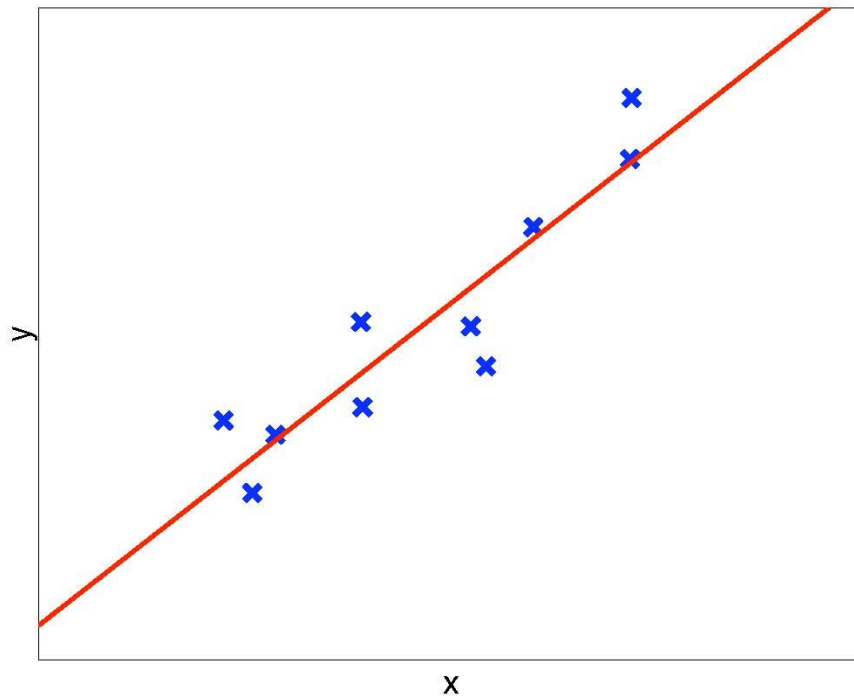
$$X^T Y =$$
$$\begin{bmatrix} 1 & 1 & 1 & 1 & 1 & 1 & 1 & 1 & 1 & 1 \\ 0.86 & 0.09 & -0.85 & 0.87 & -0.44 & -0.43 & -1.10 & 0.40 & -0.96 & 0.17 \end{bmatrix} \times \begin{bmatrix} 2.49 \\ 0.83 \\ -0.25 \\ 3.10 \\ 0.87 \\ 0.02 \\ -0.12 \\ 1.81 \\ -0.83 \\ 0.43 \end{bmatrix}$$
$$= \begin{bmatrix} 8.35 \\ 6.4601 \end{bmatrix}$$

Linear Regression

An example

$$\mathbf{w} = (X^T X)^{-1} X^T Y \approx \begin{bmatrix} 1.06 \\ 1.61 \end{bmatrix}$$

The fitted linear model is: $y \approx \mathbf{1.06} + \mathbf{1.61x}$



Linear Regression

Summary of the matrix equation

- The optimal solution can be computed in polynomial time in the size of the data set.
- Too simple for most real-valued problems
- The solution is $w = (X^T X)^{-1} X^T Y$, where
 - X is the data matrix, augmented with a column of 1's Y is the column vector of target outputs
 - A very rare case in which an analytical exact solution is possible
 - Nice math, closed-form formula, unique global optimum

Linear Regression

Problems with $w = (X^T X)^{-1} X^T Y$

- 1. High Computational Cost:** Matrix inversion has a complexity of $O(n^3)$, making it inefficient for datasets with many features.
- 2. Numerical Instability:** Ill-conditioned matrices (with near-zero determinants) lead to inaccurate results due to rounding errors during inversion.
- 3. Memory Intensive:** Storing and processing $X^T X$ requires large memory, especially for high-dimensional datasets.
- 4. Singularity Issues:** Multicollinearity (linearly dependent features) makes $X^T X$ non-invertible, requiring additional techniques like regularization.
- 5. Scalability Problems:** Closed-form solutions don't scale well to big data, making iterative methods like **Gradient Descent** more practical and efficient.

Linear Regression

An alternative to $(X^T X)$: Gradient descent

Let $\hat{y}_i = w_0 + w_1 x_{i,1}$ be the prediction for i th example input $X_i = (x_{i,1})$ with only one feature.

and y_i be the correct value for input example X_i

So, the error in prediction would be $\hat{y}_i - y_i$

The objective the best line would be to minimize the Mean Squared Error (MSE) cost function with m examples,

$$J = \frac{1}{m} \sum_{i=1}^m (\hat{y}_i - y_i)^2$$

Linear Regression

Gradient Descent for Linear Regression

A linear regression model can be **trained** using the **gradient descent** optimization algorithm, which iteratively adjusts the model's parameters to minimize the mean squared error (MSE) on the training dataset.

To update the values of w_0 and w_1 and reduce the cost function (by minimizing the RMSE), the model applies Gradient Descent.

The process begins with random values for w_0 and w_1 , and these values are progressively updated to achieve the lowest possible cost, ultimately resulting in the best-fit line.

Linear Regression

Gradient Descent for Linear Regression

A gradient is essentially a derivative that describes how slight changes in the inputs of a function affect its outputs.

If we differentiate the cost function $\mathbf{J}$ with respect to $\mathbf{w}_0$

$$\begin{aligned} J'_{w_0} &= \frac{\partial}{\partial w_0} \frac{1}{m} \sum_{i=1}^m (\hat{y}_i - y_i)^2 \\ &= \frac{1}{m} \sum_{i=1}^m 2(\hat{y}_i - y_i) \frac{\partial}{\partial w_0} (\hat{y}_i - y_i) \\ &= \frac{2}{m} \sum_{i=1}^m (\hat{y}_i - y_i) \frac{\partial}{\partial w_0} (w_0 + w_1 x_{i,1} - y_i) \\ &= \frac{2}{m} \sum_{i=1}^m (\hat{y}_i - y_i) (1 + 0 - 0) \\ &= \frac{2}{m} \sum_{i=1}^m (\hat{y}_i - y_i) \end{aligned}$$

Linear Regression

Gradient Descent for Linear Regression

If we differentiate the cost function **J** with respect to **w₁**

$$\begin{aligned} J'_{w_1} &= \frac{\partial}{\partial w_1} \frac{1}{m} \sum_{i=1}^m (\hat{y}_i - y_i)^2 \\ &= \frac{1}{m} \sum_{i=1}^m 2(\hat{y}_i - y_i) \frac{\partial}{\partial w_1} (\hat{y}_i - y_i) \\ &= \frac{2}{m} \sum_{i=1}^m (\hat{y}_i - y_i) \frac{\partial}{\partial w_1} (w_0 + w_1 x_{i,1} - y_i) \\ &= \frac{2}{m} \sum_{i=1}^m (\hat{y}_i - y_i) (0 + x_{i,1} - 0) \\ &= \frac{2}{m} \sum_{i=1}^m (\hat{y}_i - y_i) x_{i,1} \end{aligned}$$

Linear Regression

Update w_0 and w_1 values in order to reduce the Cost function

The goal of linear regression is to determine the coefficients of a linear equation that best fits the training data. This is achieved by adjusting the coefficients in the direction of the negative gradient of the Mean Squared Error with respect to those coefficients.

The respective intercept and coefficient of $\mathbf{X}$ will be adjusted by a factor of α , where α represents the learning rate.

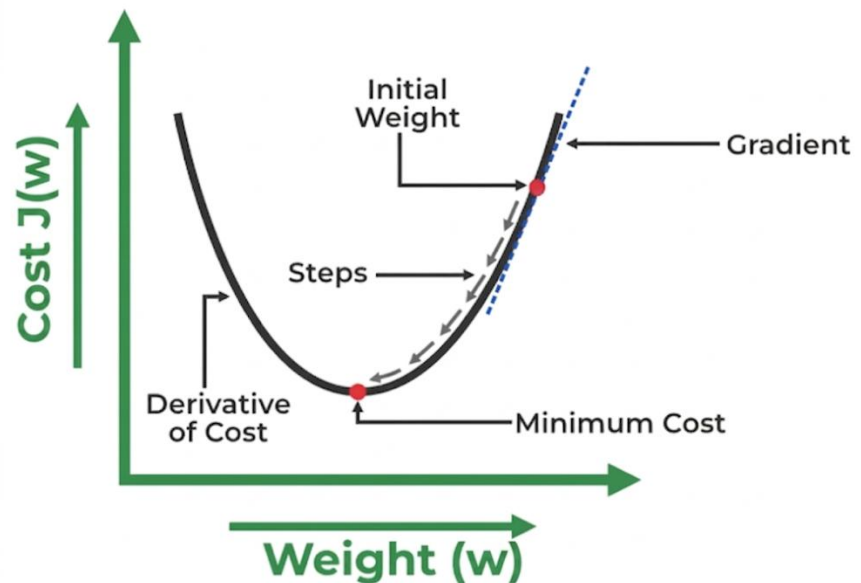
$$w_0 = w_0 - \alpha(J'_{w_0})$$

$$= w_0 - \alpha\left(\frac{2}{m} \sum_{i=1}^m (\hat{y}_i - y_i)\right)$$

$$w_1 = w_1 - \alpha(J'_{w_1})$$

$$= w_1 - \alpha\left(\frac{2}{m} \sum_{i=1}^m (\hat{y}_i - y_i)x_{i,1}\right)$$

Repeat until convergence



Linear Regression

Numerical Example

$$\hat{y}_i = w_0 + w_1 x_{i,1}$$

Let $w_0=300$, $w_1=10$

Learning rate, $\alpha = 0.0001$

$$\hat{y}_i = 300 + 10x_{i,1}$$

$$J = \frac{1}{m} \sum_{i=1}^m (\hat{y}_i - y_i)^2$$

Age (x)	Salary (y)
30	800
37	950
25	600
43	1050
50	1200
29	740
46	1100

Linear Regression

Numerical Example

$$\hat{y}_i = w_0 + w_1 x_{i,1}$$

Let $w_0=300$, $w_1=10$

Learning rate, $\alpha = 0.0001$

$$\hat{y}_i = 300 + 10x_{i,1}$$

$$J = \frac{1}{m} \sum_{i=1}^m (\hat{y}_i - y_i)^2$$

$$= \frac{1}{7} [(-200)^2 + (-280)^2 + \dots]$$

$$= 74485.71$$

$x_{i,1}$	y_i	$\hat{y}_i$	$\hat{y}_i - y_i$
30	800	600	-200
37	950	670	-280
25	600	550	-50
43	1050	730	-320
50	1200	800	-400
29	740	590	-150
46	1100	760	-340

Linear Regression

Numerical Example: Learning (updating w_0 and w_1)

$$\begin{aligned} J'_{w_0} &= \frac{2}{m} \sum_{i=1}^m (\hat{y}_i - y_i) \\ &= \frac{2}{7} (-200 - 280 - 50 \dots) \\ &= -497.14 \end{aligned}$$

$$\begin{aligned} J'_{w_1} &= \frac{2}{m} \sum_{i=1}^m (\hat{y}_i - y_i) x_{i,1} \\ &= \frac{2}{7} (-6000 - 10360 \dots) \\ &= -20388.57 \end{aligned}$$

$x_{i,1}$	y_i	$\hat{y}_i$	$\hat{y}_i - y_i$	$(\hat{y}_i - y_i)x_{i,1}$
30	800	600	-200	-6000
37	950	670	-280	-10360
25	600	550	-50	-1250
43	1050	730	-320	-13760
50	1200	800	-400	-20000
29	740	590	-150	-4350
46	1100	760	-340	-15640

Linear Regression

Numerical Example: Learning (updating w_0 and w_1)

$$\begin{aligned}w_0(\text{new}) &= w_0 - \alpha(J'_{w_0}) \\&= 300 - 0.0001 * (-497.14) \\&= \mathbf{300.0497}\end{aligned}$$

$$\begin{aligned}w_1(\text{new}) &= w_1 - \alpha(J'_{w_1}) \\&= 10 - 0.0001 * (-20388.57) \\&= \mathbf{12.03}\end{aligned}$$

Updated equation be

$$\hat{y}_i = 300.0497 + 12.03x_{i,1}$$

$$J(\text{new}) = \frac{1}{m} \sum_{i=1}^n (\hat{y}_i - y_i)^2 = 39084.83$$

$x_{i,1}$	y_i	$\hat{y}_i$	$\hat{y}_i - y_i$
30	800	660.95	-139.05
37	950	745.16	-204.84
25	600	600.80	0.80
43	1050	817.34	-232.66
50	1200	901.55	-298.45
29	740	648.92	-91.08
46	1100	853.43	-246.57



**Repeat until
convergence**

Linear Regression

Example with code

Python Notebook

https://colab.research.google.com/drive/1XIYk601MWojTZaCprZAbFKShl3aIG_e

Logistic Regression

Logistic regression (also called logit regression) is commonly used to estimate the probability that an instance belongs to a particular class (e.g., what is the probability that this email is spam?).

If the estimated probability is greater than a given threshold (typically 50%), then the model predicts that the instance belongs to that class (called the positive class, labeled “1”), and otherwise it predicts that it does not (i.e., it belongs to the negative class, labeled “0”). This makes it a binary classifier.

Just like a **linear regression** model, a **logistic regression** model computes a weighted sum of the input features (plus a bias term), but instead of outputting the result directly like the linear regression model does, it outputs the logistic of this result.

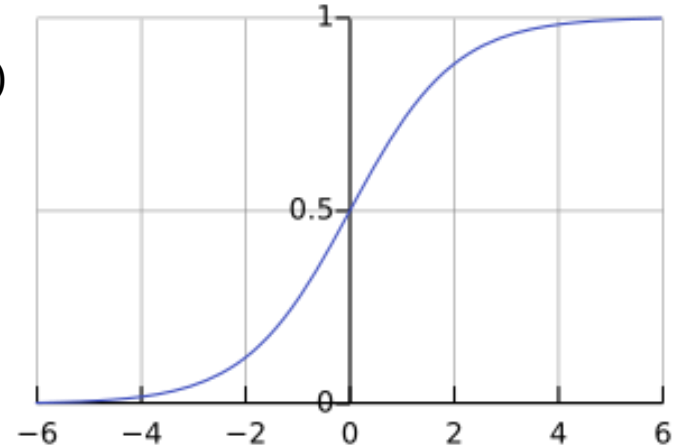
$$\sigma(t) = \frac{1}{1 + e^{-t}}$$

Logistic Regression

$\hat{y}_i = w_0 + w_1 x_{i,1}$ becomes $\hat{y}_i = \sigma(w_0 + w_1 x_{i,1})$

where $\sigma(t) = \frac{1}{1+e^{-t}}$

The function is also known as sigmoid function.



Notice that $\sigma(t) < 0.5$ when $t < 0$, and $\sigma(t) \geq 0.5$ when $t \geq 0$, so a logistic regression model using the default threshold of **50%** probability predicts **1** if $w_0 + w_1 x_{i,1}$ is positive and **0** if it $w_0 + w_1 x_{i,1}$ is negative.

In this way Logistic Regression can be used as a binary classifier.